Candidate Prompt

Overview

Build a web application that displays a carousel of movies released in a 4 week span using the provided local API.

Requirements

Functional Requirements

1. **Fetch Movie Data**
   - Use the local Movie API `GET /movies` endpoint
   - Assuming today is March 18, 2026. Filter movies to only show those with a `primary_release_date` within the past 4 weeks.
   - Use the `primary_release_date_gte` and `primary_release_date_lte` query parameters to define the date range (format: `yyyy-MM-dd`)
2. **Display Movies in a Carousel**
   - Create a responsive carousel/slider component
   - Display at least the following for each movie:
     - Movie poster image
     - Movie title
     - Release date
   - Allow navigation between movies (prev/next buttons or swipe)
   - Each movie in the carousel should be clickable to navigate to its detail page
3. **Movie Detail Page**
   - Clicking a movie in the carousel should navigate to a dedicated detail page for that movie
   - The detail page should be built using the data available from the API — you may use the list response or fetch a single movie by ID
   - The detail page should display at minimum:
     - Movie poster image
     - Title
     - Release date
     - Overview/description
     - User rating (`vote_average`)
     - Director and cast
     - Plot summary

- Feel free to incorporate any other available fields (runtime, rating, tagline, backdrop image, genres) to create a polished detail view
- The page should include a way to navigate back to the carousel

4. **Date Filtering Logic**
   - Assume today is March 18, 2026. Filter movies to only show those with a `primary_release_date` within the past 4 weeks.
     - Though not required, your application can allow the user to select any 4 week span between `Jan 9, 2026` and `May 8, 2026`
   - Pass these as `primary_release_date_gte` and `primary_release_date_lte` query parameters (NOTE: that `primary_release_date` refers to the movie's official worldwide release date, not regional or home media release dates)
   - Display the release date in a user-friendly format

Technical Requirements

- Use your preferred frontend framework/library (React, Vue, vanilla JavaScript, etc.)
- Make HTTP requests to the local Movie API
- Handle loading states and errors gracefully
- Ensure responsive design (works on mobile and desktop)

API Information

Base URL: `http://localhost:3001`

What	Example
All movies	`GET /movies`
Filter by date range	`GET /movies?primary_release_date_gte=yyyy-MM-dd&primary_release_date_lte=yyyy-MM-dd`
Single movie by ID	`GET /movies/23`
Poster image	`GET /images/23.png`

Date filter operators:

- append `_gte` (≥) or `_lte` (≤) to the field name.

Endpoints:

- `GET /movies?primary_release_date_gte=2026-03-11&primary_release_date_lte=2026-03-25`

Example API Response Structure

`/discover/movie?primary_release_date.gte=yyyy-MM-dd&primary_release_date.lte=yyyy-MM-dd`


```
{
    "id": 23,
    "title": "Project Hail Mary",
    "primary_release_date": "2026-03-20",
    "poster_path": "/images/23.png",
    "overview": "An astronaut wakes up alone on a spacecraft with no memory of who he is or how he got there…",
    "vote_average": 8.4,
    "backdrop_path": "/images/backdrop_23.jpg",
    "genres": ["Science Fiction", "Adventure", "Drama"],
    "director": "Phil Lord, Christopher Miller",
    "cast": ["Ryan Gosling", "Sandra Huller", "Milana Vayntrub"],
    "runtime": 156,
    "rating": "PG-13",
    "tagline": "Alone in space. Earth's last chance.",
    "plot_summary": "Astronaut Ryland Grace wakes up millions of miles from Earth…"
  }
```


Movie Object Fields

Field	Type	Description
`id`	number	Unique movie identifier
`title`	string	Movie title
`primary_release_date`	string	Release date (`yyyy-MM-dd`)
`poster_path`	string	Relative URL for the poster image
`overview`	string	Short synopsis
`vote_average`	number	Average user rating (0–10)
`backdrop_path`	string	Relative URL for a wider backdrop image
`genres`	string[]	Genre names (e.g. `"Action"` , `"Comedy"`)

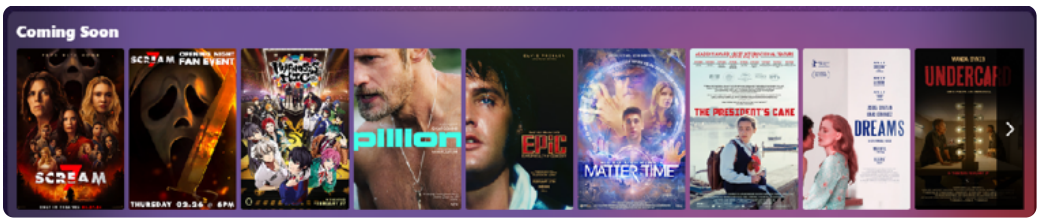
`director`	string	Director name(s)
`cast`	string[]	Top-billed cast members
`runtime`	number	Runtime in minutes
`rating`	string	MPAA rating (e.g. `"PG-13"`, `"R"`)
`tagline`	string	Short marketing tagline
`plot_summary`	string	Full plot summary for a details page

Image URLs: prepend the base URL to `poster_path` or `backdrop_path`:

```
http://localhost:3001/images/23.png
http://localhost:3001/images/backdrop_23.jpg
```

Visual Reference

Below is an example of the expected carousel layout. Your implementation does not need to match it exactly, but it should convey the same general structure — a horizontal row of movie posters with a visible navigation control and a section heading.



Key things to note:

- Movie posters are displayed side by side in a horizontal row
- A section heading (e.g. "Coming Soon") sits above the carousel
- Navigation arrow(s) allow the user to scroll through posters
- The carousel should feel like a browsable list, not a single full-screen slide

Setup Instructions

1. Extract the zip

Windows: Right-click `movie-mock-api.zip` → *Extract All* → choose a destination folder.

macOS: Double-click `movie-mock-api.zip`. A folder named `movie-mock-api` will appear beside it.

Linux:

```
unzip movie-mock-api.zip -d movie-mock-api
```

You should end up with this structure:

```
movie-mock-api/
├── db.json
└── public/
    └── images/
        ├── 1.jpg
        ├── 2.jpg
        └── ...

```

2. Start the API server

See **SETUP.md** inside the extracted folder for detailed instructions on starting the server and verifying it is running.

3. Build your app

- Create a new project with your preferred setup
- Use the local Movie API described above
- Implement the requirements
- Be prepared to explain your code and decisions